

Data Schema

Why Schema Discovery?

Handing teams a complete schema removes the part of the challenge that actually matters: can your agent figure out what it's asking for? Instead of documenting every field here, the data layer exposes a request that returns the schema at runtime. Teams call it, get back the shape of the world, and use that (plus the Glossary for the domain concepts) to construct real queries.

That being said, the schema itself is expected to stay stable for the duration of the challenge.

Structure

Calling the schema endpoint returns a description per resource: which resources exist, their fields, and each field's type. Roughly:

```
{
  "Policy": {
    "type": "object",
    "fields": {
      "id": { "type": "number" },
      "status": { "type": "string" },
      "hazard_tags": { "type": "array", "itemSchema": { "type": "string" } },
      "locations": { "type": "reference", "resource": "Location", "cardinality": "many" },
      "insured": { "type": "object", "fields": { "name": { "type": "string" }, "...": {} } }
    }
  }
}
```

- `type: "object"` has a `fields` map.
- `type: "array"` has an `itemSchema` describing each element (which may itself be an object with `fields`, or a scalar).
- `type: "reference"` points at a record in another resource. It carries a target `resource` and a `cardinality` of `"one"` or `"many"`.
- Anything else is a scalar (`string`, `number`, `boolean`, etc).

References

A reference field does not store the referenced record. It stores an **id**, or an array of ids for `"many"`.

References can optionally be `expand`ed (think “joined”) as a part of the query pipeline to hydrate them into the result set, avoiding the need to run N+1 queries if you know you’ll want that data ahead of time.